

TP – Premiers pas avec Git et GitHub

Sauvegarder et partager son travail, la bonne routine

BTS CIEL – 1^{re} année

Gestion de projet

Durée 3 h

Prérequis ouvrir un terminal, utiliser `cd` et `ls`, éditer un fichier avec `nano` ou VS Code

Matériel poste sous Ubuntu (droits `sudo` pour installer Git), navigateur web, adresse e-mail pour créer un compte GitHub

Nom :

Prénom :

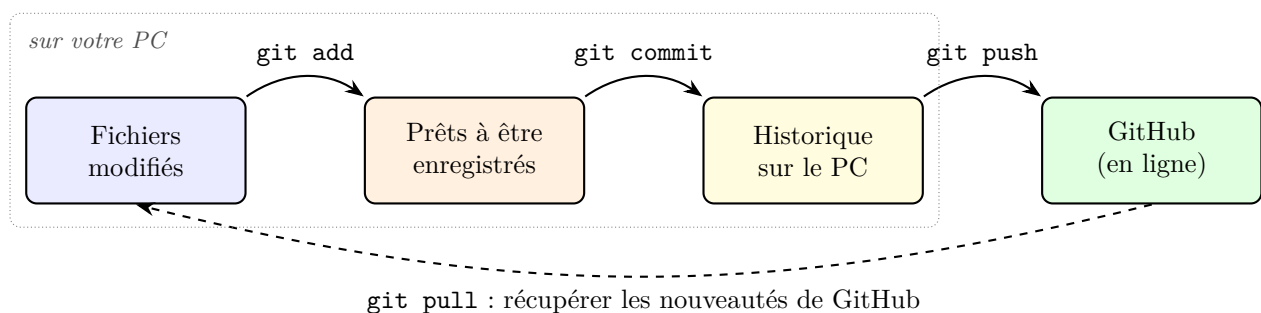
Objectif. Ce TP ne fait pas de vous des experts de Git. Il vous apprend **une routine** que vous utiliserez dans tous vos TP et dans votre projet de 2^e année :

`git pull` → travailler → `git status` → `git add` → `git commit` → `git push`

À la fin du TP, votre travail sera sauvegardé sur GitHub, avec l'historique de chaque modification.

1 Git et GitHub en deux mots

- **Git** est un logiciel installé sur votre PC. Il enregistre l'historique de vos fichiers : chaque sauvegarde s'appelle un **commit** (une « photo » de vos fichiers, avec un message et votre nom).
- **GitHub** est un site web qui héberge une copie de votre travail. Vous le retrouvez depuis n'importe quel PC, et vous pouvez le partager avec votre équipe.
- Un **dépôt** (*repository*, ou *repo*) est un dossier de projet suivi par Git. Il existe en deux exemplaires : le dépôt **distant** sur GitHub et le dépôt **local** sur votre PC.



Commit ou push ?

`git commit` enregistre une version *sur votre PC*. Tant que vous n'avez pas fait `git push`, GitHub ne la connaît pas : si le PC tombe en panne ou si vous changez de poste, elle est perdue.

2 Préparer GitHub et son poste

Exercice 1 – Créer son compte et son premier dépôt sur GitHub

1. Sur <https://github.com>, créez un compte (**Sign up**). Choisissez un nom d'utilisateur sérieux : il pourra apparaître sur votre CV. Notez-le.

.....

2. Créez un dépôt : bouton **+** en haut à droite, puis **New repository**.

— *Repository name* : **tp-git** ;

— visibilité : **Private** ;

— activez l'option qui ajoute un fichier **README** ;

— validez avec **Create repository**.

3. Que contient votre dépôt ? Cliquez sur l'horloge « **1 Commit** » : quel est le message de ce commit ?

.....

.....

Installer Git sur Ubuntu. Ouvrez un terminal (**Ctrl+Alt+T**) et vérifiez si Git est présent :

```
git --version
```

S'il répond *Command 'git' not found* (« commande introuvable »), installez-le :

```
sudo apt update
sudo apt install git
```

sudo demande le mot de passe de votre session Ubuntu. Quand vous le tapez, **rien ne s'affiche** : c'est normal, tapez-le puis **Entrée**. Vérifiez ensuite avec **git --version**.

Configurer Git. Git inscrit votre nom et votre e-mail dans chaque commit. Tapez, avec *vos* informations :

```
git config --global user.name "Prénom Nom"
git config --global user.email "adresse-du-compte-github@exemple.fr"
git config --global core.editor nano
git config --global pull.rebase false
git config --global credential.helper "cache --timeout=14400"
```

- **core.editor** et **pull.rebase** règlent le comportement de Git quand il fusionne votre travail avec celui d'un camarade (partie 7).
- **credential.helper** : Git retient votre jeton de connexion (voir ci-dessous) *en mémoire* pendant 4 heures, sans l'écrire sur le disque. Vous n'avez pas à le recoller à chaque **git push**.

Vérifiez avec **git config --global --list**.

Poste partagé

Si d'autres élèves utilisent la même session Ubuntu, vérifiez en début de séance que **git config user.name** affiche bien *votre* nom (sinon, refaites la configuration). En fin de séance, tapez **git credential-cache exit** pour que Git oublie votre jeton.

3 Se connecter à GitHub depuis le terminal : le jeton

GitHub refuse le mot de passe du compte dans le terminal. À la place, on utilise un **jeton d'accès personnel** (*personal access token*) : une longue clé secrète qui commence par `ghp_`.

Exercice 2 – Créer son jeton d'accès

1. Ouvrez directement la page des jetons : <https://github.com/settings/tokens>, puis **Generate new token** → **Generate new token (classic)**. GitHub peut vous redemander votre mot de passe.

Par les menus : photo de profil **tout en haut à droite** → **Settings**, puis tout en bas de la colonne de gauche **Developer settings** → **Personal access tokens** → **Tokens (classic)**.

Ne pas confondre

L'onglet **Settings** de la page d'un *dépôt* (menu avec **Collaborators**, **Branches**, **Webhooks**...) règle ce dépôt seulement : on n'y trouve pas **Developer settings**. Les jetons se créent dans les paramètres de votre *compte*.

2. Remplissez :
 - *Note* : `tp-ciel`;
 - *Expiration* : **90 days**;
 - cochez la case **repo** (accès à vos dépôts).

Validez avec **Generate token**.

3. **Copiez le jeton tout de suite** : GitHub ne l'affiche qu'une seule fois. Gardez-le en lieu sûr (gestionnaire de mots de passe, par exemple). Si vous le perdez, il suffit d'en générer un nouveau.
4. Pourquoi ne faut-il *jamaïs* écrire ce jeton dans un fichier de votre dépôt ?

.....
.....

4 Récupérer son dépôt sur le PC : git clone

Sur la page GitHub de votre dépôt, cliquez sur le bouton vert **Code**, onglet **HTTPS**, et copiez l'adresse. Dans le terminal Ubuntu, on colle avec **Ctrl+Maj+V** (et non Ctrl+V) :

```
cd ~  
git clone https://github.com/VOTRE-COMPTE/tp-git.git
```

Votre dépôt étant privé, Git demande de vous identifier :

```
Username for 'https://github.com': votre-nom-utilisateur-github  
Password for 'https://votre-nom-utilisateur-github@github.com': (collez le jeton)
```

Au lieu du mot de passe, **collez le jeton** (Ctrl+Maj+V). Rien ne s'affiche : c'est normal. Puis :

```
cd tp-git  
ls  
git status
```

`git clone` ne se fait **qu'une seule fois** par poste : il crée le dossier `tp-git`, relié à GitHub.

« Authentication failed » ?

Vous avez sans doute tapé le mot de passe du compte au lieu du jeton, ou fait une erreur dans le nom d'utilisateur. Recommencez la commande `git clone`.

Exercice 3 – Cloner son dépôt

1. Clonez votre dépôt avec les commandes ci-dessus. Quel(s) fichier(s) voyez-vous avec `ls` ?

.....

2. Recopiez les deux premières lignes affichées par `git status`.

.....

.....

5 La routine : status, add, commit, push

Commande	Rôle
<code>git status</code>	« Où en suis-je ? » : quels fichiers sont nouveaux, modifiés, prêts à être enregistrés. À taper tout le temps.
<code>git add fichier</code>	sélectionne un fichier pour le prochain commit (<code>git add .</code> sélectionne tout le dossier)
<code>git commit -m "message"</code>	enregistre les fichiers sélectionnés, avec un message qui dit ce qui a changé
<code>git push</code>	envoie les commits vers GitHub

Exercice 4 – Premier cycle complet (guidé)

Vous allez documenter le réseau de la salle B204.

1. Dans le dossier `tp-git`, créez le fichier `plan_adressage.txt` avec `nano plan_adressage.txt` (enregistrer : **Ctrl+O**, **Entrée**; quitter : **Ctrl+X**) :

```
Plan d'adressage - salle B204
VLAN 10  ADMIN    192.168.10.0/24  passerelle 192.168.10.254
VLAN 20  ELEVES   192.168.20.0/24  passerelle 192.168.20.254
```

2. `git status` : dans quelle rubrique, et de quelle couleur, apparaît le fichier ?

.....

3. `git add plan_adressage.txt`, puis `git status`. Qu'est-ce qui a changé ?

.....

4. Enregistrez :

```
git commit -m "Ajout du plan d'adressage de la salle B204"
git status
```

Que vous dit maintenant `git status` ? Quelle commande vous suggère-t-il ?

.....

-
5. Envoyez vers GitHub avec `git push`, puis `git status`. Rafraîchissez la page GitHub : que voyez-vous ?
-
-

Lire git status

<i>Untracked files</i> (rouge)	nouveau fichier, pas encore suivi → <code>git add</code>
<i>Changes not staged for commit</i> (rouge)	fichier modifié, pas encore sélectionné → <code>git add</code>
<i>Changes to be committed</i> (vert)	prêt à être enregistré → <code>git commit</code>
<i>Your branch is ahead of 'origin/main'</i>	commits pas encore envoyés → <code>git push</code>
<i>nothing to commit, working tree clean</i>	tout est enregistré

Exercice 5 – À vous de jouer (sans aide)

1. Ajoutez au plan d'adressage la ligne suivante :

```
VLAN 30  SERVEURS 192.168.30.0/24  passerelle 192.168.30.254
```

2. Créez un fichier `inventaire.csv` :

```
nom;type;ip
SW-B204;switch;192.168.10.2
PC-B204-01;poste;192.168.20.11
```

3. Faites **deux commits séparés** (un par modification), avec des messages clairs, puis envoyez-les sur GitHub. Notez ci-dessous toutes les commandes utilisées, dans l'ordre.
-
-
-
-
-
-
-
-

4. Sur GitHub, cliquez sur le compteur de commits. Combien y en a-t-il ?
-

Voir l'historique. Sur le PC, `git log --oneline` affiche un commit par ligne, du plus récent au plus ancien. Sur GitHub, le compteur de commits donne la même liste. En cliquant sur un commit, on voit les lignes ajoutées (en vert) et supprimées (en rouge).

Exercice 6 – Bon ou mauvais message ?

Un bon message dit *ce qui a changé*, en une phrase courte. Pour chaque message, indiquez B (bon) ou M (mauvais).

Message	B/M
1 « modif »	
2 « Ajout du VLAN 30 (serveurs) au plan d'adressage »	
3 « azerty »	
4 « Correction de l'adresse IP du switch dans l'inventaire »	
5 « travail de mardi »	

6 Récupérer les nouveautés : git pull

Quand le dépôt GitHub a changé (modification par un camarade, ou par vous depuis un autre PC), on récupère ces changements avec `git pull`.

Exercice 7 – Modifier sur GitHub, récupérer sur le PC

1. Sur GitHub, ouvrez `README.md`, cliquez sur le crayon (**Edit this file**) et ajoutez la ligne : *Documentation du réseau de la salle B204 – Prénom Nom*. Validez avec **Commit changes**.
2. Sur le PC, tapez `git status`. Git signale-t-il la modification faite sur GitHub ?
.....
3. Tapez `git pull`, puis affichez le README avec `cat README.md`.
.....
4. Pourquoi `git status` ne voyait-il pas la modification à la question 2 ?
.....
.....

Jamais de mot de passe sur GitHub

Tout ce qui est poussé sur GitHub reste dans l'historique, même si l'on supprime le fichier ensuite. Ne versionnez jamais un mot de passe, une clé ou une configuration contenant un secret. Si cela arrive, le mot de passe doit être considéré comme compromis et changé.

7 Travailler à deux sur le même dépôt

Exercice 8 – En binôme (élèves A et B)

1. **(A)** Sur la page GitHub de votre dépôt : **Settings** → **Collaborators** → **Add people**, et ajoutez le compte de B. **(B)** Acceptez l'invitation reçue par e-mail (ou sur <https://github.com/notifications>).

2. **(B)** Clonez le dépôt de A dans un nouveau dossier :

```
cd ~
git clone https://github.com/COMPTE-DE-A/tp-git.git tp-git-A
cd tp-git-A
```

3. **(B)** Ajoutez la ligne `SW-B204-2;switch;192.168.10.3` à la fin de `inventaire.csv`, puis faites la routine jusqu'au `git push`.

4. **(A)** Faites `git pull` et vérifiez que la ligne de B est bien dans votre inventaire.

.....

5. **En même temps** : A ajoute une ligne au README, B ajoute un VLAN au plan d'adressage. Chacun fait `add` et `commit`. B fait `git push` en premier, *puis* A. Que se passe-t-il pour A ?

.....

.....

6. **(A)** Faites `git pull`. Si `nano` s'ouvre avec un message de fusion, enregistrez (**Ctrl+O**, **Entrée**) et quittez (**Ctrl+X**). Puis `git push`. Tout le monde fait ensuite `git pull` : les deux dépôts sont-ils identiques ?

.....

« push rejected » : pas de panique

Si `git push` est refusé, c'est que GitHub a des nouveautés que vous n'avez pas. Faites `git pull`, puis `git push`. Si Git parle de *CONFLICT*, appelez l'enseignant.

8 Défi final (en autonomie)

Sans regarder les pages précédentes, en vous aidant seulement de l'aide-mémoire :

1. Créez sur GitHub un nouveau dépôt privé `mes-tp-ciel`, avec un README, et clonez-le.
2. Ajoutez un fichier `compte_rendu.md` contenant le titre de ce TP et trois choses que vous avez apprises. Commit et push.
3. Complétez le README avec votre nom et votre classe. Commit et push.
4. Modifiez `compte_rendu.md` directement sur GitHub, puis récupérez la modification sur le PC.
5. Montrez à l'enseignant la liste des commits sur GitHub et un `git status` « propre ».

Ce dépôt vous servira pour ranger vos prochains TP.

Auto-évaluation

Je sais. . .	Pas encore	En partie	Oui
expliquer la différence entre Git et GitHub, entre commit et push	☐	☐	☐
créer un dépôt sur GitHub et le cloner sur un PC	☐	☐	☐
lire <code>git status</code> et savoir quelle commande taper ensuite	☐	☐	☐
enregistrer mon travail : <code>add</code> , <code>commit</code> avec un message clair	☐	☐	☐
envoyer et récupérer mon travail : <code>push</code> , <code>pull</code>	☐	☐	☐
réagir à un <i>push rejected</i>	☐	☐	☐

Aide-mémoire

La routine de chaque séance

- | | |
|---|--|
| 1. <code>git pull</code> | récupérer les nouveautés (début de séance) |
| 2. <i>travailler</i> | créer, modifier des fichiers |
| 3. <code>git status</code> | voir ce qui a changé |
| 4. <code>git add fichier</code> | sélectionner (ou <code>git add .</code> pour tout) |
| 5. <code>git commit -m "message"</code> | enregistrer sur le PC |
| 6. <code>git push</code> | envoyer sur GitHub (au plus tard en fin de séance) |

Une seule fois

<code>git config --global user.name "Nom"</code>	votre nom (dans chaque commit)
<code>git config --global user.email "mail"</code>	votre e-mail (celui du compte GitHub)
<code>git clone <url></code>	copier un dépôt GitHub sur le PC

Pour s'y retrouver

<code>git status</code>	où en suis-je ?
<code>git log --oneline</code>	liste des commits

Pour aller plus loin

- S. Chacon et B. Straub, *Pro Git*, 2^e éd., Apress, 2014. Libre, traduit en français : <https://git-scm.com/book/fr/v2> (chapitres 1 et 2).
- Documentation GitHub en français : <https://docs.github.com/fr/get-started>.